

Fantasy Premier League Points Forecasting and Squad Optimization

Project status report

Generated 2026-07-04

Abstract. This report summarizes the current state of a Fantasy Premier League (FPL) points-prediction and squad-optimization system, originally developed as a Master’s thesis (LSTM forecasting in R combined with a hand-written MILP squad optimizer) and since rewritten into a single, weekly-runnable Python pipeline. It documents the pipeline’s architecture, the evaluation methodology used to validate changes, the results of forecasting-technique experiments conducted against the production system (all reported honestly, including negative results), and two exploratory branches investigating alternative approaches. History now spans six seasons (2020-21 through 2025-26); the current 6-model ensemble improves on the original LSTM system by approximately 29% in realized squad points over an equivalent 31-gameweek backtest window. The most recent addition - fixture-difficulty and minutes-projection features plus a probabilistic (quantile) forecasting module - improved forecast accuracy at every position but **regressed** realized squad points, an unresolved, honestly-reported finding discussed in Section 5.4.

1 Introduction

This project predicts Fantasy Premier League (FPL) player points and uses those predictions to recommend weekly squad decisions (transfers, starting lineup, captaincy, and chip usage). It began as a Master’s thesis combining an LSTM forecasting model (implemented in R/Keras) with a mixed-integer linear program (MILP) for squad selection, based on the formulation of Kristiansen et al. (2018). The system has since been rewritten into a single Python pipeline (the `fpl` package) so that it can be run weekly during a live season rather than only as a one-off academic validation.

This report documents the system’s current architecture, the empirical work done to validate and harden it, and the outcome of several forecasting-technique experiments run against the production pipeline. Where an experiment failed to improve on the existing system, that is reported explicitly - the guiding principle for this phase of the project was to test candidate improvements honestly rather than assume a “more sophisticated” or “more principled” technique must help.

2 Data

Historical data is sourced from Vaastav Anand’s public FPL data repository (`vaastav/Fantasy-Premier-League` on GitHub), which mirrors the official FPL API’s per-gameweek player statistics

back to the 2016-17 season. This project originally used four seasons (2022-23 onward); history was subsequently extended back to **2020-21**, giving six seasons at the time of writing (2020-21 through 2025-26, 162,981 player-gameweek rows). The extension was a clear win (Section 5.2) and cost only the Opta expected-goals family and the `starts` column for the two oldest seasons (not available in `merged_gw.csv` before 2022-23; encoded as missing, which LightGBM handles natively).

Gameweeks are indexed with a single ascending counter (`GW_global`) across all seasons rather than resetting to 1 each season, since a raw `GW` value is ambiguous without knowing which season it belongs to. Season N occupies gameweeks $(N - 1) \times 38 + 1$ through $N \times 38$; with the current six-season history, the 2024-25 validation window used throughout this report (season gameweeks 1-31) is GW153-183.

2.1 Exploratory data analysis, all six seasons

`notebooks/eda.ipynb` covers the distribution of `total_points` by position and season across the full six-season history. The key finding, consistent with domain expectations, is that `total_points` is heavily right-skewed and zero-inflated: most player-gameweek observations score between 0 and 2 points, with an occasional large-value outlier from a goal, hat-trick, or high bonus-point haul. This distributional shape - not merely a stylistic preference - is the reason the mean absolute scaled error (MASE), rather than mean absolute error (MAE) alone, is used to judge forecast quality throughout this project (see Section 3.4).

Descriptive statistics of `total_points` for players who featured (`minutes` ≥ 1), per season and position, show the distribution has been broadly stable across all six seasons - no season stands out as an outlier, and medians are flat at 1-2 points everywhere:

Season	GK mean	DEF mean	MID mean	FWD mean	n (all positions)
2020-21	3.75	3.00	2.88	3.18	10393
2021-22	3.59	2.96	2.92	3.04	10461
2022-23	3.61	2.63	2.74	3.00	11345
2023-24	3.19	2.42	2.81	3.14	11384
2024-25	3.31	2.43	2.69	3.21	11566
2025-26	3.34	3.05	2.91	2.95	11498

Mean `total_points`, players with `minutes` ≥ 1 , per season/position. Full descriptive statistics (count/mean/median/std/min/max) and per-season `boxplots/KDE/QQ` plots are in `notebooks/eda.ipynb`.

Formal tests confirm what the plots show. A Shapiro-Wilk test rejects normality decisively at every position (starters only, $p < 0.001$ throughout) - motivating MASE over MAE and the choice of tree/linear models over anything assuming Gaussian errors. An Augmented Dickey-Fuller test on the average points per gameweek per position is more mixed with six seasons of history than it was with four: GK, MID, and FWD reject the unit-root null (stationary, $p < 0.05$), but **DEF does not** ($p = 0.41$) - the average defender score-per-gameweek series has drifted enough over six seasons (plausibly the 2025-26 introduction of `defensive_contribution` points, which disproportionately affects DEF/MID scoring) that a constant mean can no longer be assumed for that position over the full history. This is a refinement to note, not an alarm: it doesn't change any modeling choice already made (rolling-window features and per-position models already adapt to a drifting mean by construction), but it is a reason to treat any classical

fixed-mean time-series baseline (AR(1), ARIMA) at DEF with more caution than the other three positions.

3 Forecasting methodology

3.1 Feature engineering

Features are shifted rolling-window and season-to-date statistics per player (3- and 5-gameweek rolling means, and an expanding season-to-date mean, over roughly twenty per-gameweek statistics including points, minutes, expected goals/assists, bonus points system score, and market value). Every feature is shifted by one gameweek relative to its target row, so that no row’s features can see that row’s own outcome - a standard no-leakage discipline for supervised time-series forecasting.

Two further feature groups were added most recently:

- **Fixture difficulty.** The official FPL Fixture Difficulty Rating (FDR, 1-5) merged onto every player row by (team, gameweek): `fixture_difficulty` (the opponent faced that gameweek) and `fixture_difficulty_next3` (mean FDR across this fixture and the next two scheduled - an “easy run of fixtures” signal). These are **not** shifted: fixture lists are published ahead of a gameweek being played, so a model forecasting gameweek t genuinely knows the opponents at $t, t + 1, t + 2$ without any leakage.
- **Minutes projection (“nailedness”).** `start_rate_roll5` and `mins60_rate_roll5` - rolling 5-game rates of starting and of a full 60+-minute appearance, both shifted one gameweek like the other rolling features. A player who won’t start scores approximately zero regardless of ability, so projecting playing time from recent starts is one of the most predictive signals in FPL forecasting generally.

3.2 Per-position ensemble

Rather than a single global model, four independent models are trained - one per playing position (goalkeeper, defender, midfielder, forward) - since the statistics that matter for predicting a goalkeeper’s points (saves, clean sheets) are largely disjoint from those that matter for a forward’s (shots, expected goals), and pooling all positions into one model risks one position’s scale dominating the loss function.

At each position, six computationally cheap model types are trained and compared: LightGBM (gradient-boosted trees), Ridge regression, ElasticNet, Random Forest, Extra Trees, and k -nearest neighbors. Tree-based models receive raw features including missing values (a player’s first few gameweeks have no rolling history yet), since LightGBM and the scikit-learn tree ensembles handle missing values without imputation. Linear and distance-based models are wrapped in a median imputer and standard scaler, since those estimators require complete, scaled input.

The six models’ predictions are blended into a single ensemble per position using non-negative least squares (NNLS) regression, which finds non-negative weights (summing to one) minimizing squared error against the true outcome. Critically, these blend weights are fit on one half of a held-out validation window and the ensemble’s reported accuracy is evaluated on the **other** half - a genuine holdout, ensuring the reported ensemble accuracy is not simply the result of fitting weights and evaluating them on the same rows used to choose them.

3.3 Evaluation methodology

Two complementary evaluation approaches are used:

1. **Static split:** models are trained on gameweeks up to a cutoff and evaluated on a fixed, contiguous held-out window immediately following it - specifically the same GW77-107 window (2024-25 season, gameweeks 1-31) that the original LSTM system was validated on, so the two approaches are directly comparable.
2. **Walk-forward (expanding-window) validation:** for each gameweek from a starting point onward, a model is trained on strictly earlier data and evaluated only on that single gameweek, then the training window expands by one gameweek and the process repeats. This gives a far larger number of genuinely out-of-sample evaluation points than a single fixed split, at the cost of retraining many times.

3.4 Error metrics

Mean absolute error (MAE) alone is difficult to interpret for an intermittent series like FPL points: an MAE of 1.2 looks identical whether the underlying target typically scores 8 points or typically scores 0. The mean absolute scaled error (MASE; Hyndman & Koehler, 2006) addresses this by expressing forecast error relative to a naive one-gameweek-lag benchmark, with the benchmark’s own error computed strictly on the training window (so the scale itself cannot leak information from the evaluation window it is used to judge). A MASE below 1 indicates the model outperforms the naive “same as last gameweek” forecast; a MASE above 1 indicates it does not, independent of the target’s raw point-scale.

4 Squad optimization

Given a predictions CSV, `fpl/milp/optimize.py` recommends a squad using a mixed-integer linear program closely following the formulation of Kristiansen et al. (2018), which is also the lineage underlying Venter & van Vuuren (2024) - a paper reviewed in full during this project (see References). The model is re-solved every gameweek over a configurable lookahead horizon in a rolling-horizon fashion, with only the first gameweek’s decision locked in before rolling forward. Constraints include:

- a 15-player squad of exactly 2 goalkeepers, 5 defenders, 5 midfielders, and 3 forwards;
- a total squad value not exceeding a fixed budget;
- no more than 3 players from any single real-world club;
- an 11-player starting lineup satisfying formation constraints (exactly 1 goalkeeper, at least 3 defenders, at least 1 forward);
- a captain (2x points) and vice-captain, with automatic substitution when a starter is unused;
- a limited number of free transfers per gameweek, with a 4-point penalty per additional transfer;
- wildcard, free-hit, bench-boost, and triple-captain chip logic, each usable at most once (twice for wildcard) per season and mutually exclusive within a gameweek.

The objective maximizes the sum of predicted lineup points plus captain bonus, less any transfer penalties incurred, summed across the lookahead horizon.

5 Results

5.1 Backtesting reference point

The pipeline was validated against the original LSTM+R system by running both through the same MILP optimizer on an equivalent 31-gameweek backtest window (2024-25 season, gameweeks 1-31 - originally GW77-107 under the four-season history, now GW153-183 under

the current six-season history), using actual realized points rather than forecasts to score the resulting squads:

System	Actual points, 2024-25 GW1-31
Old LSTM + MILP	1526
New LightGBM + MILP (4-season history)	1811
New 6-model ensemble + MILP (4-season history)	1900
New 6-model ensemble + MILP (6-season history)	1966
1. fixture/minutes features + probabilistic module	1880 ¹

The 6-model ensemble on the extended 6-season history improves on the original LSTM system by approximately 29% in realized squad points over an equivalent evaluation window and identical optimizer - the project’s best validated configuration to date. The most recent feature additions improved forecast accuracy but reduced this number; see Section 5.4 before treating them as a net win.

5.2 History extension: 2022-23 -> 2020-21

`config.DEFAULT_START_SEASON` was extended from 2022-23 back to 2020-21 after confirming empirically (checking `vaastav`’s raw column headers season by season) that the key power predictors (`bps`, `ict_index`, `influence`, `creativity`, `threat`, `position`, `team`) are available that far back; only the Opta expected-goals family and `starts` are lost for the two oldest seasons. Dataset size grew from 113k to 163k rows.

Position	Ensemble MASE, 4 seasons	Ensemble MASE, 6 seasons	OLS index, 6 seasons
GK	0.637	0.595	0.579 ²
DEF	0.874	0.830	0.842
MID	0.871	0.811	0.869
FWD	1.067	0.984	1.056

MASE, 2024-25 GW1-31 static split. FWD dropped below 1.0 (beats the naive forecast) for the first time across every experiment run in this project.

This translated directly into realized points: the same backtest scored **1966, up from 1900 (+3.5%)** - confirmation that the MASE improvement was not just a metric artifact.

5.3 Forecasting-technique experiments

Following a full reading of Venter & van Vuuren (2024) - whose §4 MILP formulation was found to match the existing optimizer almost exactly, confirming no further optimizer work was warranted - project focus shifted entirely to forecasting quality. Venter & van Vuuren’s own case study credits full-season lookahead and forecast quality, not MILP sophistication, for their result (top 4.08% of roughly 8.24 million FPL managers in the 2020-21 season using a broadly similar optimizer). Several additional forecasting techniques flagged by that paper as relatively strong individual performers were implemented and tested against the production ensemble, honestly and without assuming a “more principled” technique must help. This comparison was run on the four-season history (GW77-107) before the extension in Section 5.2, so the ensemble/

¹Unresolved regression despite improved MASE - see Section 5.4.

²The one case where the plain OLS index beats the ensemble - see Section 7.

OLS columns below are superseded by the 6-season numbers above - the point of this table is the **relative** ranking of simple techniques against each other and against the ensemble, which is not expected to change with more history:

Position	Ensemble	OLS (index)	Baseline ³	SES	Theta	Croston	AR(1)	ARIMA
GK	0.64	0.66	0.69	0.68	0.70	0.89	0.87	0.86
DEF	0.87	0.90	0.94	0.90	0.91	1.06	1.06	1.10
MID	0.87	0.93	0.98	0.94	0.95	1.10	1.11	1.14
FWD	1.07	1.16	1.13	1.11	1.14	1.39	1.29	1.36

MASE, GW77-107 static split (4-season history). Lower is better; bold marks the ensemble, the best performer at every position in this comparison.

Naive drift and Holt’s linear trend were also tested (see `RESEARCH_LOG.md` for full figures) and likewise underperformed the existing baseline everywhere. Across all seven simple techniques tested: **simple exponential smoothing (SES) is the best of the simple baselines, but nothing tested beats the full ensemble at any position.** Croston’s method - specifically designed for intermittent-demand series and flagged in the source literature as a strong performer - was the worst performer here, because pooling it across an entire position (rather than selecting it only for the specific players whose scoring pattern it suits, as the source literature actually did) hides the per-player heterogeneity that made it work in the original study. This motivated a subsequent planning exercise into per-player model selection (Section 6.2), rather than per-position pooling, as the more promising remaining direction.

Plain OLS regression as the designated index. On the recommendation that a simple, unregularized multiple linear regression should serve as the project’s designated benchmark - “index” - that every other technique must clear, exactly as a passive market index is the bar an active strategy has to beat, plain OLS regression on the full approximately 70-feature set was added to the comparison (`fpl/model/models.py`). It turns out to be a genuinely strong baseline: better than every time-series technique in the table above at every position, trailing only the regularized linear models (Ridge, ElasticNet) by a small margin - expected, since the engineered feature set includes many mutually correlated rolling-window statistics that unregularized OLS cannot down-weight the way Ridge/ElasticNet can. The production ensemble beats this index at every position in this comparison (by 0.02-0.09 MASE) - reassuring, since it confirms the ensemble’s added complexity is earning its keep against a genuinely competitive simple benchmark, not merely against a weak straw man. (On the extended 6-season history, OLS goes on to edge the ensemble specifically at GK - see Section 5.2 and Section 7.)

5.4 Fixture difficulty, minutes projection, and probabilistic forecasting

Three additions were made together: fixture-difficulty and fixture-window features, minutes-projection (“nailedness”) features, and a separate probabilistic (quantile regression) forecasting module producing a p10/p50/p90 interval per player-gameweek instead of a single number (motivated by captaincy: two players with equal expected points are not equal decisions once the 2x captain multiplier rewards upside).

³3-gameweek rolling mean, falling back to season-to-date mean.

Position	Ensemble MASE, before	+ fixture + minutes
GK	0.595	0.588
DEF	0.830	0.799
MID	0.811	0.799
FWD	0.984	0.959

MASE, 2024-25 GW1-31 static split, 6-season history. DEF improved most, consistent with fixture difficulty mattering most for clean-sheet-dependent scoring.

Forecast accuracy improved at every position, and the ensemble still beat the OLS index everywhere. **However, the actual-points backtest regressed from 1966 to 1880 (-86 points, -4.4%) on the same window** - a genuine, important divergence between forecast accuracy and realized outcome. The leading hypothesis is that fixture-difficulty features make the mean forecast smoother and more regressed-to-the-mean, nudging the MILP (which maximizes **expected** points and is blind to variance) toward safe, nailed players and away from the high-ceiling differentials that actually haul on the captain multiplier. This is precisely the gap the new probabilistic module exists to close, and it is why the actual-points backtest, not MASE, remains this project’s gold-standard check (see `CLAUDE.md`). **These features are committed but not yet declared a net win** - resolving the divergence (checking whether it is noise on a second window, or feeding the probabilistic p90/upside signal into captain selection) is the highest-priority open item, tracked in `TODO.md`.

The probabilistic module’s [p10, p90] interval coverage came out at 0.88-0.93 against an ideal 0.80

- the intervals are somewhat wider than a perfectly calibrated 80% band, most likely because the

p10 quantile pins near 0 for blank-prone players in a zero-inflated target; FWD was best-calibrated at 0.88. Usable as-is for relative risk ranking; a conformal-prediction calibration pass is a possible follow-up.

5.5 Optimizer constraint testing

A regression test (`tests/test_optimize_constraints.py`) verifies that the MILP optimizer’s output always satisfies its own constraints - squad composition, budget, club limits, and starting-lineup formation - using a small synthetic predictions dataset. This is the project’s first automated test; it was confirmed to actually catch regressions by temporarily loosening a constraint in the optimizer and observing the test fail before reverting the change.

6 Parallel exploratory branches

Two lines of investigation were pursued on isolated git branches, deliberately separated from the production pipeline so that speculative or higher-risk work could not entangle or block it.

6.1 Bayesian belief-state MDP manager

Matthews, Ramchurn & Chalkiadakis (2012) frame FPL as a belief-state Markov decision process, maintaining Bayesian beliefs over each player’s starting probability, goal probability, and assist probability, updated via conjugate-prior rules as results are observed, with team selection posed as a multi-dimensional knapsack problem and solved via either a myopic (single-gameweek) policy or a Q-learning policy with a bounded candidate-action pool. An

implementation of this approach (`bayesian_manager/`, on branch `experimental/bayesian-mdp-manager`) was backtested on the identical GW77-107 window:

Manager	Actual points, GW77-107
Myopic (single-gameweek) policy	1083
Q-learning candidate-pool policy	847
Production ensemble + MILP (reference)	1900

Both variants score well below the production system, which is expected given several necessary simplifications relative to the source paper: a proxy for player-absence detection (using zero minutes rather than a curated injury/suspension list), a simplified discrete-time club-level goal model in place of the paper’s continuous-time birth process, no simulation of bonus points, cards, or saves, and a one-step-lookahead approximation in place of full multi-step search. This branch is not merged and does not affect the production pipeline.

6.2 Per-player forecasting model selection

Venter & van Vuuren (2024) select, for each individual player, whichever of roughly fifteen candidate forecasting methods (including several tested in Section 3.4’s table) performs best for that specific player by cross-validated MASE - rather than pooling one method across an entire position, as this project’s own experiments above did. A planning exercise (not an implementation) was conducted to assess the feasibility of this approach for this project’s data (`PER_PLAYER_MODEL_SELECTION_PLAN.md`, branch `worktree-agent-afeff933546ce7d37`). Its key finding: approximately 35% of players active in the current season have no prior-season history at all, and the median prior history among players who do have some is only 38 gameweeks - capping the technique’s plausible reach at roughly 60-65% of the live player pool even before results are considered, since per-player cross-validated model selection is unreliable on very short series. The plan’s recommendation is a cautious, pilot-scale go (a single-position pilot before any larger commitment), noting it is plausible - and would itself be a legitimate, cheap-to-detect negative result - that per-player selection mostly just re-selects the existing ensemble anyway, consistent with the pattern established across Section 3.4’s experiments.

7 Discussion and limitations

The forecasting experiments conducted so far establish a consistent pattern: the existing 6-model, per-position ensemble outperforms every simpler alternative tested at every position, with one exception. This is a meaningful finding in itself - it means further effort spent adding individual “more sophisticated” model types to the existing architecture is unlikely to yield further gains, and that any future improvement is more likely to come from a genuine architectural change (such as per-player rather than per-position model selection) than from adding another candidate model type to the existing pool.

The exception, introduced by the six-season history extension, is **goalkeeper**: plain OLS regression (MASE 0.579) now slightly beats the 6-model ensemble (0.595) - the first position where the simple index wins. A plausible explanation is that GK is a small, simple-signal position (saves, clean sheets, few confounding statistics) that benefits more from the extra two seasons of raw data volume than from ensemble complexity, which may be overfitting a small/simple signal. Worth a cheap follow-up check on the GK ensemble’s NNLS blend weights (`TOD0.md`).

The forward (FWD) position remains the hardest to forecast in absolute terms (highest MASE at every stage of this project), consistent with forwards having the highest-variance, most

“boom-or-bust” scoring profile of the four positions - though it is also the position that improved most from added history, dropping below the naive-forecast benchmark ($\text{MASE} < 1$) for the first time once six seasons were available.

The six-season extension also surfaced a **defender non-stationarity** finding in the EDA (Data section): the average DEF score-per-gameweek series no longer rejects the Augmented Dickey-Fuller unit-root null over the full six seasons, plausibly reflecting the 2025-26 **defensive_contribution** scoring-rule change. This doesn’t invalidate any modeling choice already made (the rolling/shifted features already adapt to a drifting mean), but is a reason for caution around any future fixed-mean time-series baseline at DEF specifically.

The most significant open finding is the **MASE-vs-points divergence** introduced by the fixture/minutes features (Section 5.4): forecast accuracy improved at every position, yet the squads built from those forecasts scored fewer real points. This is the clearest demonstration yet of why this project treats the actual-points backtest, not MASE, as the gold-standard check - a model can become more accurate in aggregate while becoming worse at the specific job (identifying high-ceiling captaincy differentials) that most affects the score. Resolving it is the top priority before any further forecasting work.

A known limitation of the live weekly driver (`fpl/run_week.py`) is that its fixture and current-squad fetching from the official FPL API can only be exercised end-to-end once a season is set up on the FPL website (typically a few weeks before gameweek 1); this had not been tested live as of this writing, since the 2026-27 season had not yet opened.

8 Conclusion and future work

This project has been rewritten from a one-off Master’s thesis validation into a maintainable, weekly-runnable Python pipeline that measurably outperforms its LSTM-based predecessor (an approximately 29% improvement in realized backtest points, on the current six-season history). This phase of work added a proper error metric (MASE), a regression test suite, tested seven alternative forecasting techniques against the production system (honestly reporting that none beat it), extended the training history from four to six seasons (a clear win), and added fixture- difficulty, minutes-projection, and probabilistic (quantile) forecasting - the last of which improved accuracy but regressed realized points, an open and unresolved finding. Two exploratory branches (a Bayesian belief-state MDP manager and a per-player model-selection plan) remain available for future investigation without having touched the production pipeline.

The most pressing concrete next step is resolving the MASE-vs-points divergence from the fixture/minutes/probabilistic work: determine whether the -86 -point regression is noise (re-run the backtest on a second window), whether feeding the probabilistic module’s upside signal into captain selection recovers it, or whether the features should be reverted if neither resolves it. After that, a small-scale pilot of per-player forecasting model selection at a single position remains the most promising longer-term direction, to test - cheaply - whether the heterogeneity that this project’s position-pooled experiments could not exploit is in fact present and exploitable at the individual-player level.

9 References

- Croston, J. D. (1972). Forecasting and stock control for intermittent demands. *Journal of the Operational Research Society*, 23(3), 289-303.
- Hyndman, R. J., & Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4), 679-688.
- Kristiansen, B. K., Gupta, A., & Eilertsen, W. (2018). *Developing a forecast-based optimisation model for Fantasy Premier League* (MSc thesis). Norwegian University of Science and Technology, Trondheim.
- Matthews, T., Ramchurn, S. D., & Chalkiadakis, G. (2012). Competing with humans at fantasy football: Team formation in large partially-observable domains. *Proceedings of the Twenty-Sixth AAAI Conference on Artificial Intelligence*, 1394-1400.
- Venter, V., & van Vuuren, J. H. (2024). An optimisation approach towards soccer Fantasy Premiere League team selection. *ORiON*, 40(1), 69-107.